SPECTRAVIDEO

USERS GROUP WELLINGTON

THE LAST NEWSLETTER

(EVER)

SPECTRAVIDEO USERS GROUP

WELLINGTON

Greetings ! (again)

The more astute among you i.e thoe ones who can read, will have noticed that the last newsletter was full of goodies. But those goodies were largely reproduced from the previous issue ! (lots of red faces and blushes from our obviously overworked editor)

This letter and what follows is by way of compensation for this cock up and also to give you the very last of the articles which we had prepared for the newsletter. Included is all kinds of good stuff in no particular order. I feel this will help us to go out with a bang instead of a wimper. (can't spell either)

Included in this "issue" is a list of the books and newsletters we have in our library. After some constenation as to what to do with this we decided to break it up and offer the books to you all. This decision resulted in a howl of protest and as a result John Hodgson has paid us $140 for the lot to keep it together. This really is a bargain price but still very generous of John. He has further offered to allow us to lend anything. The lending will be handled by Scotty Baxter and anyone wishing to to borrow books etc can send him a cheque for $15 plus $2.50 postage and he will send you the book for a six week period. If he gets the book back in that time he will send you back your $15 cheque. His address can be found in the library listing.

The money paid for the library will be used to help pay for membership to Osborne etc for those who write to us expressing some interest as laid out in the April newsletter.

We still have quite a few copies of the back issues of our newsletter and we have decided to offer these to you FREE ! There are some gaps in the back numbers but if are in early with your return postage and bubble bags ($1.50 - $2.00 and 300mm X 250mm bags please) to Jim Oliver at 14 Cherry blossom Grove Lower Hutt you should end up with a near complete set if you let know which ones you want.

The last matter to bring up is this : I has been elected onto the Osborne group's committee to look after the SV members interests. I will also be looking after the entire basic and CP/M P.D library (some 400 discs in all). Any person elected to the Osborne committee is expected to take out membership to FOG (First Osborne Group USA). This entitles me to a copy of the FOG CP/M newsletter each month. The catch is that this membership costs about $70.00 so anyone wishing to borrow this excellent newsletter will be required to assist me financially. Some Wellington members have agreed to do this already especially since the SV club is assisting in a major way with subs as already mentioned. We must now support the Osborne group and best way to do this is to in turn support the USA group whose continued existance is essential if we are to continue to have access to the wealth of high class , well organinsed CP/M and MS-DOS P.D that is available.

Well thats it from me as chairman of SVUG Wellington, however I will be only too pleased to assist any continuing members in any way I can, so keep in touch if you are confused, worried, or need PD help.

It's been hard work but worth it.

SPECTRAVIDEO USERS GROUP WELLINGTON

CONTENTS :

BOOK REVIEW

Forecasting On Your Microcomputer
By Daniel B.Nickell : Pub. TAB (2nd. ED 1988)

This most unusual and intereresting book has hundreds of IBM Microsoft Basic (IBM BASICA) programs which are easily converted to SV. The title is a little misleading since the book contains chapters on the following :

Forecasting Philosophies	: Correlation Analysis
Database management	: Economic Analysis
Numerical Analysis	: Survey / Opinion Analysis
Astromical Applications	: Population Analysis
Random Event Forecasting	: Simulations
Game Analysis	: Regression Analysis
Probabilty Analysis	: Summation Rules

There are 385 pages absolutely stacked with high class programs and background info on the above and many more applications. Some of the programs are presented in Symphony spreadsheet format. There is info on how to convert these to dBASE III+ format.

The book is highly technical in parts and the maths used in these areas assumes a fairly advanced understanding. However don't let this put you off since there are many parts of the book where almost anyone can learn new ideas of programming to encompase the ideas presented. For example how would you write a Basic program to best "fit" any given data to an equation ? This sort of thing is easily understood after reading the correct chapter.

All the programs contain printer based commands in the form of easy to understand LPRINTs for output to printers if desired. None of these is machine specific and are applicable to our systems. You will get the most from this book if you have an 80 Column card fitted in which case the only change normally necessary is to swap the LOCATE variables since IBM BASICA uses the syntax LOCATE ROW,COLUMN (SV uses COLUMN,ROW). Most of the programs can be run using SVMBASIC. The ones containing graphic

statements such as LINE etc are exactly the same as SV.

Since the programs in the book were originally written for a TRS 80, the variable names are designed for first two letter significance. This avoids the normally tedious conversions so our systems will recognise the difference between, say, YEAR1$ and YEAR2$

There is miles too much in this book to do it justice in a short review and my recommendation to you is to get a copy from the library (mine from Lower Hutt) and have a read. If you are in the least bit interested in ANY serious application for your micro there will be something in this book for you.

There are good suggested reading lists and exercises when you've finished trying out all the programs ! The programs are all available on a disc from the publishers. If anyone buys such a disc we can arrange to convert these (at no cost) to SV format.

DISC FILES : USING INPUT AND OUTPUT

By Jim Oliver

One of the most confusing things for me when I first started using discs was the proper creation and use of disc files. Not program files but writing and reading ASCII information to or from a disc.

In SV disc basic there are 2 types of files. There are sequential files and random access files.

In this article I'm going to cover sequential files only.

Sequential files are data files which must be accessed from the beginning and continue to the end. Random access files are those which can be accessed from any place in the file i.e bits of info can be grabbed from the middle or the end or wherever without having to start at the beginning. It is therefore fairly slow to get desired data from sequential files compared to random files but unless the files are large (> 20K) sequential are easier to use and understand.

So remember, all that follows is for sequential files.

Firstly to create a disc file try this :

```
Ex.1    10 OPEN "1:TESTFL.1" FOR OUTPUT AS #1
        20 CLOSE
```

If you now have a look at your files, you should have an extra one called TESTFL.1 with nothing in it i.e pretty useless.

Now try this :

```
Ex.2    10 CLEAR 1000          'clear 500 bytes of work space
        20 OPEN "1:TESTFL.1" FOR OUTPUT AS #1:'Note OUTPUT
        30 INPUT "PLEASE TYPE YOUR NAME ";NM$
```

```
        40 PRINT #1,NM$:     'print info to disc file
        50 INPUT "TODAYS DATE";DT$
        60 PRINT #1,DT$
        70 CLOSE #1:         'Or just CLOSE
```

It's not essential to CLEAR space for this program to work but good practice to do this for later and larger files.

Now, to see what's in the file we do this :

```
Ex.3    10 OPEN "1:TESTFL.1" FOR INPUT AS #1:'Note INPUT
        20 IF EOF(1) THEN CLOSE:END
        30 INPUT #1,A$:' Input from file to RAM
        40 PRINT A$:    ' Print to screen
        50 GOTO 20:     'Is there any more data in this file ?
```

You should now be starting to see what this is all about. I'm going to assume that you will use your disc basic reference book to look up the meaning of the syntax used above. However there is quite a bit the books don't tell you.

The INPUT or OUTPUT statements that read or create the files are from the point of view of the RAM. A file opened for OUTPUT is one which is created on the disc, then requires some OUTPUT from RAM by way of input statements from the keyboard or other device to begin to fill it up. The files opened for INPUT are READ from the disc INTO memory in preparation to then be displayed on the screen or printer or used in some other way. Simple huh ?

After OPENING a file the MOST IMPORTANT thing to remember after you have finished using it is to CLOSE by using CLOSE (file No.) or just CLOSE which closes ALL files opened. This CLOSING of files is very important. If this is not done you will probably ruin your disc with the dreaded BAD ALLOCATION TABLE error. This can be very hard or impossible to fix. Hint : type KEY 1,"CLOSE:FILES" then use this key a lot just to make sure all files are definately, well and truly CLOSED and all the disc info well written to the proper tables on the disc !

The system allows a maximum number of OPEN files at once. The default is one but by using the MAXFILES statement many more (15) can be in use or OPEN at once. If three files need to be opened, use MAXFILES=3 as the FIRST line of the program. The number after the 'AS' is the number which identifies the file to be read or written to. For example :

```
Ex.4    10 MAXFILES=3:CLEAR 1000:'Always MAXFILES first
        20 OPEN "1:TESTFL.1" FOR INPUT AS #1:'Already on disc
        30 OPEN "1:TESTFL.2" FOR OUTPUT AS#2
        40 OPEN "1:TESTFL.3" FOR OUTPUT AS#3
        50 INPUT #1,A$:'Any string name is OK
        60 PRINT #2,A$:'Write A$ to TESTFL.2
        70 PRINT #3,A$:'Write A$ to TESTFL.3
        80 CLOSE:      'ALL files closed
```

Notice I didn't use the EOF (end of file) statement above since I knew that TESTFL.1 had at least two 'bits' of data in it i.e name then date separated

in the file by carrage return and line feed characters. In Ex.3 above I used EOF(file No) to detect the end of the file so I didn't get the error INPUT PAST END. This is a fairly harmless error but can be easily avoided by testing for end of file BEFORE inputting more data from the file.

You should now have three files on disc. The first one TESTFL.1 will have your name and a date, and the others will have your name only in them. By using FOR / NEXTs and INPUTs you can input all sorts of info to the file. You can even input control characters if you should ever want to do that !

There are many very useful things which can be done by using sequential files. Quite often when pulling a program to bits I want to know the line number where a particular GOSUB or GOTO (or whatever) is mentioned.

Firstly save the file to disc in ASCII mode (SAVE "1:file",A) then use something like this :

```
10 CLEAR 1000  :X$="GOSUB1000"
20 OPEN "1:FILE" FOR INPUT AS #1
25 IF EOF(1) THEN CLOSE:END:  'Always CLOSE after EOF
30 INPUT #1,A$ :'Input a complete program line
40 IF INSTR(A$,X$) > 0 THEN PRINT A$
50 GOTO 25
```

What this does is automatically inputs each program line in turn (line 30) then finds out if the desired string (GOSUB1000) is included in the line, and if it is (INSTR > 0) then print it out to the screen (or LPRINT for printer). If it is not in that line then go back and input a new line and re-search for GOSUB1000. Note : if the program lines contain commas you must use LINE INPUT in place of INPUT since input stops at commas whereas LINE INPUT stops at carrage returns. This process continues until the end of the file is reached or you press control+ STOP.

If you do stop the program before it reaches the EOF marker, you must remember to type CLOSE:FILES. If you don't then change discs you will most definately get a BAD ALLOCATION ERROR. In fact it's worth forcing this error in this way just to see what happens. But please, only use a disc you are prepared to FORMAT soon after !

One thing I haven't mentioned is this : when a file is OPENed for OUTPUT everything in that file at this time is lost. Remember Ex.1 where we created a new disc file ? If TESTFL.1 had had lots of data in it, it would all be gone if we ran Ex.1. This is the same as SAVING a program with nothing in it under the same name as an existing good program file. So... be careful here.

There is a way around this by using the command APPEND as in :

```
OPEN "1:TESTFL.1" FOR APPEND AS #1
```

This does not then destroy any data already in TESTFL.1 but APPENDS data to the end of the existing file. However, I will let you work on this one yourself. Try it, it's easy !

There are many uses to which disc files can be put. To save on memory for a

program you can put all your SPRITE data into a separate file then read it only when needed. Often the data for adventure games is stored in disc files.

If a player is asked to input some data to for an adventure game, this data can be written to a disc file for later use by a separate program. This gets around the problem of passing variables to new program. (SV basic does not have commands like CHAIN or COMMON.) The game "so far" can be saved to disc for use later to "load a saved game"

Phone lists which sort or search are easy stuff using disc files, disc catalogue programs must use disc files. etc etc.

Rather than reinvent the wheel I will give below the program which some of you may have on your system disc. If you have .. fine, if you have not then get typing. It is called PLIST and it has nothing to do with drinking. See the end of these articles for the program listing.
(SVPHONE.BAS)

IPL SECTOR SETUP

When using SV disc basic and after "booting" or starting up your disc drive the system looks at a special place on the disc called the IPL sector at (track 20 sector 14) for any data. Usually this will be in the form of an instruction to the computer to run a particular program. For example, if you had a program called "demo" on a disc which you wished the computer to automatically load and then run after booting up the drive you would type :

```
IPL "RUN"+CHR$(34)+"1:demo"
```

This instruction will write some data at sector 14, track 20. The next time you boot this disc (this must be a "system" disc) the computer will load and run "demo". To remove this automatic feature simply type:

```
IPL ""
```

The IPL sector is also used to SET the disc status from READ ONLY (R/O) to READ/WRITE (R/W). If you type:

```
SET 1,"P"
```

the disc is set to R/O or (P)rotected status. To remove this (i.e to R/W) type:

```
SET 1,""
```

You can type a "2" for number two drive.

You all know that a sector can hold up to 255 bytes of data and the IPL sector is no exception. This means that this sector can acheive quite a lot of work WITHOUT TAKING UP ANY EXTRA disc space.

Michael Warwick from Tokoroa has written the following program which will enable you to automatically set up your computer to your own requirements. This program collects the info you input and then writes it all to the IPL

sector.

See the end for the program listing. (IPL_ED.BAS)

Copying System Tracks on Basic Discs

Recently we were lucky to have Steve Lacey from Wanganui at one of our meetings. He gave us the program below as very fast and reliable Basic disc system copy using a two disc system.

```
100 CLEAR 1000
110 PRINT"This program will copy the sysgen"
120 PRINT"on drive A, and copy it onto drive B"
140 PRINT
150 PRINT"Push ENTER to start, ESC to abort";
160 IF INPUT$(1)<>CHR$(13) THEN END
170 PRINT
180 FIELD #0, 128 AS A$, 128 AS B$
190 FOR T=0 TO 2
200 PRINT"Track:"CHR$(T+48)" ";
210  FOR S=1 TO 17-(T=0)
220   PRINT".";
230   C$=DSKI$(1,T,S)
240   DSKO$ 2,T,S
250  NEXT
260  PRINT
270 NEXT
```

Clever eh ?

Brain Teaser #3 Answer

The answer to this brain teaser was quite easy since we have a command called "MOD" (short for MODULUS) as part of our Operating System. What MOD does is returns a remainder after carrying out a divison. For example 5 MOD 2=1 or 12 MOD 6=0. C=A MOD 8 means store the remainder of the A divided by 8 in C. Get it ?

```
10 REM Brainteasers #3
20 N1=1731:N2=5363:N3=7179:N4=9903
30 FOR T=1 TO 1731
40 R1=N1 MOD T:R2=N2 MOD T:R3=N3 MOD T:R4=N4 MOD T
50 IF R1=R2 AND R2=R3 AND R3=R4 THEN PRINT T;
60 NEXT T
```

Line 20 contains the four numbers and lines 30 - 60 tries out all the possibles and then prints the result if all the remanders are equal. The FOR / NEXT loop could have counted backwards so as to find out the largest number first, however FOR / NEXT loops are not very fast when counting

backwards.

Lester Reilly from Christchurch has sent in this version of the above program. It's more fancy than mine and comes up with the same answer !

```
50 A=1731:B=5363:C=7179:D=9903
60 LOCATE,,0:SCREEN,0
100 FORZ=1731 TO 1 STEP-1
110 U=A MOD Z
120 V=B MOD Z:IF V<>V THEN 220:REM Try changing 220 to 230
130 W=C MOD Z:IF V<>W THEN 220:REM          "
140 X=D MOD Z:IF V<>X THEN 220:REM          "
150 PRINT" NUMBER = ";Z;"REMAINDER = ";U
160 PRINT"CHECK":E=FIX( A/Z):F=FIX(B/Z):G=FIX(C/Z):H=FIX(D/Z)
170 PRINT E"*"Z"+"U"=";E*Z+U
180 PRINT F"*"Z"+"U"=";F*Z+U
190 PRINT G"*"Z"+"U"=";G*Z+U
200 PRINT H"*"Z"+"U"=";H*Z+U
210  LOCATE,,1:SCREEN,1:STOP
220 PRINTZ
230 NEXTZ
240 END
```

Have you found a better way ? Please let us know if you have.

BACKING UP MACHINE CODE UNDER MSX by Jim Oliver

What follows explains a small program which sneaked into the sept. Oct issue of the newsletter without any description.

Do you remember a small but very useful basic + M/C program by D.Kelly in the December 1985 issue of the newsletter ?

I'll explain : To BSAVE any machine code to tape (or disc) the correct syntax is as follows ; BSAVE "file",start address, end address, execution address. Often the last address is the same as the first and can be left out if this is so.

The first 6 bytes written on the tape are these three addresses. Now , if you want to BSAVE any machine code to tape from memory, for example to back up a machine code program you may have, you must first find out the addresses so as to use the correct syntax. Unfortunately you can't do it from basic withput using some machine code.

Thats where Mr. Kelly's program comes in. For the full explaination of it's use and structure I refer you to the December 1985 issue.

That version only worked with SV saved tapes. Gary Clark from Auckland has put in some sweat on your behalf and now converted this program for use with MSX tapes. This is very useful with SV when using the MSX conversion software now available as many of the new MSX games are M/C.

Now you can make those all important back ups of your favourite games !

p.s If after you have BLOADED your M/C tape under MSX you type : PRINT PEEK (64703) + 256 * PEEK (64704), this will give you the START address ONLY of any BLOADED M/C. Does anyone out there know the equivalent PEEK under SV ??? Did you also know that there is another version of the SV tape header reader on the PD disc No 20 ?

UNDERSTANDING MATHS IN MACHINE CODE

BY STEVE LACEY

Over the next few articles I hope to to explain some of the fundimentals behind "number crunching" in machine code.

Even minor maths can make a machine code project seem difficult if not impossible (I once felt this way about graphic mandelbrot programs). If you understand BASIC and have used Z80 machine code, then take my hand, I will guide you through the maths related functions we have in BASIC, with short, simple, and fast subroutines. I will cover everything from number base conversion to square roots, through to graphics, and...lets find out!

1. NUMBER BASE CONVERSION

The first step toward understanding number systems is to play with conversion between decimal, (base 10) and other bases. The following BASIC program will convert integer decimal numbers into the binary (base 2). Type it in, have a play, this is an easy and effective way to learn.

```
10 INPUT" Type in a decimal number " ;A
20 If A<0 THEN PRINT" Sorry, no negatives":END
30 G$=""
40 B=INT (A/2)
50 IF A=B*2 THEN G$="0" ELSE G$="1"+G$
60 A=B
70 IF A>0 THEN 40
80 PRINT G$
```

Of course we could have used the BIN$ command, but it has a very limited range, and would not show the technique involved. That same technique works for any other base, including hexadecimal (base 16)

```
10 INPUT" Type in a decimal number" ; A
20 IF A<0 THEN PRINT" Sorry, no negatives" : END
30 G$=""
40 B=INT(A/16)
50 C=A-B*16
60 G$=MID$("0123456789ABCDEF",C+1,1)+G$
70 A=B
80 IF A>0 THEN 40
90 PRINT G$
```

We can go one step further, by extending past the decimal point

```
10 INPUT" Type in a decimal number";A
20 IF A<0 THEN PRINT" Sorry, no negatives": END
```

```
30 G$="":A=A*16777216
40 FOR X=1 TO 12
50 B=INT(A/16)
60 C=A-B*16
70 G$=MID$ ("0123456789ABCDEF",C+1,1_+G$
80 A=B
90 NEXT X
100 PRINT LEFT$(G$,6)  "."  RIGHT$ (G$,6)
```

Line 30 introduces an offset of 16^6. This pushes some of the digits on the right of the decimal point leftwards far enought to be picked up by the basic routine. This offset is removed after the operation by inserting a decimal point (hexamal point?) six places in from the right hand side. Understand? It is a bit like multiplying a decimal number by 1000, then shifting the decimal point three places to the left.

BINARY TO DECIMAL CONVERSION

Just as with the decimal system, each digit represents a different value, which increases toward the left, and increases toward the right.

DECIMAL	BINARY	HEXADECIMAL
10^2 0	2^2 1	16^2 1
10^1 0	2^1 0	16^1 A
10^0 3	2^0 1	16^0 5
10^-1 1	2^-1 1	16^-1 2
10^-2 4	2^-2 1	16^-2 7
10^-3 1	2^-3 0	16^-3 3
10^-4 6	2^-4 1	16^-4 B

Study the following program

```
10 PRINT" Type in a binary number";
20 G=0:H=1
30 A$=INPUT$ (1) : PRINT A$
40 IF A$="0" THEN G=G*2+0
50 IF A$="1" THEN G=G*2+1
60 IF A$="." THEN 90
70 IF A$=CHR(13) THEN 140
80 GOTO 30
90 A$=INPUT$(1) :PRINT A$;
100 IF A$="0" THEN H=H/2
110 IF A$="1" THEN H=H/2:G=G+H
120 IF A$=CHR$(13) THEN 140
130 GOTO 90
140 Print:PRINT G
```

This program converts a binary number into decimal as it is typed in. Again it is easily converted to suit other bases.

NEGATIVE NUMBERS

To achieve this we employ a process known as "two's compliment". It can be very confusing, particularly for those unfamiliar with binary,

To help overcome this, we will first study the same principle in decimal.

Imagine that you have a decimal system, and a limit of any two digits to play with. This allows a maximum of 100 combinations (thats 10^2) which could literally represent values from 0 to 99. Alternatively we could use those 100 combinations to represent values from -50 to 49.

Let us take the number 01, and find its negative. Take each digit and find its opposite by substracting it from 9 ie 9-0=9 & 9-1=8. Our 01 has become 98. Next we add 1 to the least significant digit, so our 01 becomes 99, ie 99 represents the value -1. Take the number 35. Invert it to make 64, add 1 to make 65, 65 is the opposite of 35. If we put 65 back through the process, it is returned to 35. The opposite of zero is zero - 0 = 0. I hear you say "HOW USEFUL"?

Now let me point out a couple of things;

1. 100-35=65 & 100-1=99
2. Any number with a first digit of 5,6,7,8, or 9 will represent a negative value.

Ok, lets try some maths. Assume a computer where the only maths functions was addition, and you wanted to work out what 32-1 was. We know that 32-1 is the same as 32+(-1). We also know that 99 represents -1. 32 + 99 = 31 (ignore the overflow)
This very same system applies to all other number bases. Let us use 8 bit binary. To invert any number, simply toggle each bit (ie make all the 0's into 1's, and vice versa) thus 01011000 becomes 10100111. Now add 1, so the negative of 01011000 is 10101000. simple huh?

Take note of these points;
If the leftmost digit is a 1, the value is negative.
Our 8 bit number can represent numbers from -128 (10000000) to 127 (01111111)

2. NUMBER FORMATS

There are three basic ways of storing a number in memory, I will briefly explain them:

BCD This stands for Binary Coded Decimal. It is frequently used, and stores two decimal digits per byte. With the help of DAA, and RRD instructions, the Z80 will use this format directly. The constant pi would show up in a hex dump like this; 03 14 15 92 (the decimal point is assumed)

BINARY The number is stored in true binary form, and is most easily manipulated by the Z80 in mathematical operations.
The constant pi would show up in a hex dump like this: 00 00 03 24 3F 6A (Assumed decimal point)

EXPONENT This covers a very wide range of methods. Generally one byte is dedicated to working as a multipler for the other bytes, just like the 6 in this scientific form: 1.538 x 10^6. These formats extend the range dramatically, by comprising accuracy. They can however, prove awkward for

simple operations such as addition. BASIC uses formats fitting this category.

THE FORMAT CHOSEN

The format I decided to use had to be easily understood, and efficient. I chose a binary format 6 bytes long, that is three each side of the binaral point. The specifications are like this:

MAXIMUM VALUE	8388607
MINIMUM VALUE	-8388608
MINIMUM ABSOLUTE VALUE	0.0000000596

I would say this is adequate for most applications.

The maths routines that follow will assume that the IX and IY registers first byte of the six byte variables involved. The result of the operation will land in the memory pointed to by the HL register pair eg (HL)=(IX)+(IY)

The following BASIC program demonstrates how a binary number could be extracted from memory location &H9000

```
10 A=PEEK(&H9000)*65536
20 B=PEEK(&H9001)*256
30 C=PEEK(&H9002)
40 D=PEEK(&H9003)/256
50 E=PEEK(&H9004)/65536
60 F=PEEK(&H9005)/16777216
70 G=A+B+C+D+E+F
80 IF G>8388607 THEN G=G-16777216
90 PRINT G
```

Now that we have a number format to work with, we will write the simplest operation; finding to two's compliment of the number pointed to by IX You will remember that the two's compliment function involves toggling each bit, then adding 1 to the least significant digit.

There are two ways of toggling each bit in A register. They are : XOR 255, and CPL. The first part could go like this.

```
NEGIX:  LD A, (IX+0)     ; Compliment (IX+0)
        CPL
        LD (IX+0),A
        LD A,(IX+1)      ; Compliment (IX+1)
        CPL
        LD (IX+1),A
        LD A, (IX+2)...
```

This would continue through to (IX+5). Part two of the routine would add 1 to (IX+5), and carry any overflow to the more significant bytes;

```
        LD A (IX+5)      ; Inc (IX+5)
        ADD A, 01
        LD (IX+5),A
```

```
        LD A,(IX+4)      ; Add carry to (IX+4)
        ADC A,00
        LD (IX+4),A
        LD A,(IX+3)..
```

Again this would continue down to (IX+0). As you can see the routine accesses each byte directly. It is very fast, but unnecessarily long (91 bytes). By using a loop we can achieve the same result in only 21 bytes.

```
NEGIX:  PUSH BC          ;(IX)=0-(IX)
        LD BC,0006       ;length of number
        ADD IX,BC        ;IX=END+1
        SCF              ;carry = 1
        LD B,06          ;Loop 6 times
NEGIXL: DEC IX           ;work back
        LD A,(IX+0)
        CPL              ;Toggle
        ADC A,0          ;Add the carry bit
        DJNZ NEGIXL      ;Loop back
        POP BC
        RET
```

Note that this routine only works because the instructions DEC IX, CPL, and DJNZ have no effect on the carry bit. Note also that on return from the routine, the only register that has been altered is the accumulator. The following two routines have the same effect on the variables pointed to by IY, and HL.

```
NEGIY:  PUSH BC          ; (IY)=0-(IY)
        LD BC,0006       ;Length of number
        ADD IY,BC        ;IY =END+1
        SCF              ;Carry = 1
        LD B,06          ;Loop 6 TIMES
NEGIYL: DEC IY           ; Work back
        LD A,(IY+0)
        CPL              ; Toggle
        ADC A,0          ; Add the carry bit
        DJNZ NEGIYL      ; Loop back
        POP BC
        RET
NEGHL:  PUSH BC          ;(HL)=0-(HL)
        LD BC,0006       ;Length of number
        ADD HL,BC        ;HL = END+1
        SCF              ;Carry = 1
        LD B,06          ;Loop 6 times
NEGHLL: DEC HL           ;Work back
        LD A,(HL)
        CPL              ;Toggle
        ADC A,0          ;Add the carry bit
        DJNZ NEGHLL      ;Loop back
        POP BC
        RET
```

NEGIX, AND NEGHL are the first in a library of subroutines. They will form

part of a base, upon which more powerfull routines will be built.

CP/M Public Domain Update

Since the first meeting at the Central Region Osborne Users Group I have had a chance to check out a few of the CP/M P.D gems from their library. I used the FOG-CPM data base to browse through the 160 odd discs to find anything useful. The following is a run down on some good stuff.

VDE25.COM and associated files.

This is a word processor written by Eric Meyer in 1987. (Yes thats 1987 !) It is a much improved version of the VDO series which some of you may already know. The VDO series are really only editors NOT word processors. VDE includes most of the requirements of a good editor and word processor. Among many of the usual requirements has a powerful macro feature which can be used to automate many of those tedious Wordstar functions, it has an undelete function. It will read and write pure ASCII files, Wordstar, and non document. The file is only 12 K and edits only from RAM ; very fast uses files up to 50 K in length. The excellent DOC file is 40 K ! I have installed it for our terminal with little problem. Absolutely the best piece of free software I've seen in years. It is available in MS-DOS as well.

EDFILE3.COM + DOC files

This is a file "doctor". It operates on all types of files, DOC and COM etc. It beats using DDT any day. The file bytes may be edited on screen and then written to the disc. This one is for the more experienced user and is highly recommended. Installs (eventually) to our terminal.

PRN.COM + DOC files

This is a Flashprint type PD program. It takes the place of the "print" part of Wordstar. It offers the ability to print any font (you design your own and save these to a file) on a standard Epson printer. It will print graphics from Wordstar files with no problems. The DOC file can be printed from PRN and will demonstrate all it's capabilities. Installs OK.

ANYCODE + DOC files

Another Flashprint type file. It will enable you tp print ANY code to a printer using Wordstar. The ASM file is used to patch WS so that code surrounded with a special character is sent to the printer. Very useful and FREE !

HARDSOFT.COM + DOC files

This is a file which removes carriage returns from the end of lines and thus makes those lines more easily formatted etc by Wordstar. It will also harden (add CR's) to WS files so they can be transmitted by modem more safely. It is lightning fast and beats everything else hands down. There's nothing more frustrating than trying to format pure ASCII lifes with WS !

There are of course MANY more files which are potentially interesting and useful to you all so ..... it's up to you.

p.s we have a compiled version of LIFE which works very fast under CP/M and is quite fascinating. It's called LIFE9.COM and is available from Jim Oliver.

BASIC LISTINGS

What follows are some of the best Basic programes to come out to SV PD for a long while plus the listings refered to above in the articles. All of these programs are available from the P.D library (plus more) at the usual charges from Jim Oliver.

The first one is a file which will print out an ASCII saved program in three columns so as to save space on your paper. It's called SVFMT.PRT.

The second is DISKED.BAS. This is an 80 column disk doctor and is based loosely on the EDFILE3.COM file mentioned above. Bob Curwood's 80 column peek / poke routines are used here (as well as a few others).

Next follows a bunch of Fractal graphics type files, very topical and all written by (as are 1 & 2 above) Steve Lacey. We are very greatful for his efforts.

Here's those basic listings I was talking about. When you look at these ponder for a moment the work that has gone in here for your benefit. Then enjoy them !

```
10 REM SPECTRA-PHONE NOTE
20 REM uses DISK
30 REM see above text for DOC
40 REM Jul 16,1983
50 REM   Clear some string space
60 MAXFILES=1:CLEAR 1000 'set file
__ buffer space and string space
70 DEFINT A-Z
80 ON ERROR GOTO 810
90 CLS
100 LOCATE 0,0:PRINT "File? ":PRINT
__ :PRINT "Default  <CR> = 1:PHONE.
__ DAT"
110 PRINT "or 1: to read files in d
__ rive 1"
120 PRINT "or 2: to read files in d

__ rive 2"
130 LOCATE 7,0,1 ' use locate to cl
__ ear the cursor so you don't see
__ a white thing flying all over th
__ e screen
140 LINE INPUT F$
150 LOCATE ,,0
160 REM check for file to use
170 REM if no file name then set f$
__   to          1:PHONE.DAT
180 IF LEN(F$)=0 THEN F$ = "1:PHONE
__ .DAT"
190 IF F$ = "1:" OR F$ = "2:" THEN
__ LOCATE 0,6:PRINT"Directory of "F
__ $:LOCATE 0,8:FILES VAL(F$):GOTO

__ 100
200 IF MID$(F$,2,1)<>":" THEN ERROR

__   0 '  force an error when this i
__ s  not  a     disk file.
210 REM we now have file name, OPEN
__   it
220 OPEN F$ FOR INPUT AS #1
230 REM OPEN file f$ as a read only


__   file and designate it as channe
__ l number 1.


240 INPUT #1,A
360 PRINT "3.    Search a name"
370 LOCATE 8,11,0
380 PRINT "4.    Delete a name"
390 LOCATE 8,13,0
400 PRINT "5.    END program and sav
__ e."
410 LOCATE 8,15,0
420 PRINT "6.    Quit."
430 LOCATE 8,19,1
440 INPUT "Enter a function number"
__ ;F:LOCATE ,,0
450 REM check for correct input
460 IF F<1 OR F>6 THEN BEEP:LOCATE
__ 8,16:PRINTSPACE$(20):GOTO 430
470 IF A=0 THEN IF F=1 OR F=3 OR F=
__ 4 THEN PRINT "          phone book
__ empty!":BEEP:GOTO 430
480 ON F GOSUB 500,560,610,660,720,
__ 790
490 GOTO 290
500  REM  list function
510 CLS:FOR I = 1 TO A
520 PRINT I;N$(I);TAB(18);P$(I)
530 NEXT
540 PRINT:PRINT:PRINT "Hit any key
__ when done__.";
550 A$ = INPUT$(1):RETURN
560 REM  Add a name
570 CLS:LOCATE 8,5,1
580 PRINT "Name: ";:LINE INPUT N$
590 LOCATE 8,7:PRINT"Phone: ";:LINE
__   INPUT P$:IF N$="" OR P$="" THEN
__   BEEP:BEEP:RETURN
600 LOCATE 8,12:INPUT "Is this corr
__   ect? ";A$:IF A$="Y" OR A$ = "y"
__   THEN A=A+1:N$(A) = N$:P$(A) = P$
__   :RETURN ELSE 570
610 REM Search function
620 CLS
630 LOCATE 8,5,1:LINE INPUT"Enter t
__ he name:";N$:I=1
640 IF N$=N$(I) THEN LOCATE 8,7:PRI
__ NT"phone # is :"; P$(I): LOCATE 3,
__ 12:PRINT"Hit any key when done__."
__ :A$=INPUT$(1):RETURN
650 I=I+1:IF I>A THEN LOCATE 8,7:PR
__ INT"Name not found!":LOCATE 3,12:
__ PRINT"Hit any key when done__.":A$
__ =INPUT$(1):RETURN ELSE 640
660 REM Delete a name
```

```
250 DIM N$(50),P$(50):IF A=0 THEN 2
__ 90


260 FOR I = 1 TO A
270 INPUT #1, N$(I),P$(I)

280 NEXT
290 CLOSE:CLS:LOCATE,,1
300 LOCATE 8,5,0
310 REM  print out function menu
320 PRINT "1.    List the telephone
__ book"

330 LOCATE 8,7,0
340 PRINT "2.    Enter a name and ph
__ one#"
350 LOCATE 8,9,0
760 PRINT #1,N$(I);",";P$(I)
770 NEXT
780 CLOSE:CLS:RUN
790 LOCATE,,1:CLS:END
800 LOCATE,,1:CLS:END
810 REM
820 REM error handler routines
830 IF ERR = 53 AND ERL = 220 THEN
__ CLS:LOCATE 0,4,1:PRINT F$" is no
__ t in the directory":INPUT "Creat
__ e it? ";A$: IF A$="Y" OR A$="y" THEN OPEN F$ FOR OUTPUT AS 1: PRINT #1,0
__ : PRINT #1, "dummy,dummy": CLOSE: RESUME 90 ELSE RESUME 90.
100 'DISC EDITOR FOR SV-328 (80 COL
__ S ONLY)
110 'USE HEX OR ASCII INPUT DIRECT
__ TO SCREEN IN EDIT MODE
120 'CURSOR IS CONTROLLED INSIDE SC
__ REEN FEILDS.HEX AND ASCII
130 'ALTERED AUTOMATICALLY:USE ] KE
__ Y TO LOCATE DIRECTORY TRACKS
140 'WORKS WITH BASIC OR CP/M DISCS
__   USE CURSOR KEYS TO CHANGE TRACK
__ S,SECTORS
150 'EXIT WITH ESCAPE OR CTRL C
160 'STEVE LACEY DID THE WORK , JIM
__   OLIVER THE PRODDING. WSVG FEB 1
__ 989
170 CLEAR 1000:DEFFNG(T,S)=(T*17+S)
__ /4 - 13:S=1:DR=1
180 ON STOP GOSUB 550:STOP ON
190 ON ERROR GOTO 550
200 SCREEN 0,1:LOCATE ,,0:CLS
205 LOCATE 25,4:PRINT"WHICH DRIVE T
670 CLS:LOCATE 8,5,1:INPUT"enter a
__ person number:";P:IF P>A THEN LOCAT
__ E 8,7:PRINT"No such person!!!":LOC
__ ATE2,13:PRINT"Hit any key to get to
__ ":LOCATE 2,15,0:PRINT"the function
__   menu__.":A$=INPUT$(1):RETURN
680 LOCATE 8,7:PRINT"The name you w
__ ant to delete is":LOCATE 8,9:PRINT
__ ":  "N$(P)
690 LOCATE 8,11,1:INPUT"Is that cor
__ rect? ";A$
700 IF A$="y" OR A$="Y"OR A$="yes"O
__ RA$="YES" THEN 710 ELSE RETURN
710 IF A=P THEN A=A-1 ELSE FOR I=PT
__ OA-1:N$(I)=N$(I+1):P$(I)=P$(I+1):
__ NEXT:A=A-1:RETURN
720 REM End and Save
730 OPEN F$ FOR OUTPUT AS 1
740 PRINT #1,A
750 FOR I = 1 TO A
760 PRINT #1,N$(I);",";P$(I)
770 NEXT
780 CLOSE:CLS:RUN
790 LOCATE,,1:CLS:END
800 LOCATE,,1:CLS:END
810 REM
820 REM error handler routines
830 IF ERR = 53 AND ERL = 220 THEN
__ BEEP:LOCATE 0,20:PRINTCHR$(27)+"
__ p"+" Bad filename, please re-ent
__ er "+CHR$(27)+"q":RESUME 100
860 LOCATE,,1:ON ERROR GOTO 0
320 'BLOAD"1:HD.MAC":DATA AT END IS
__   MACHINE CODE
330 KEY 1,"  change drive
340 KEY 2,"     edit
350 KEY 3,"     write
360 KEY 4,"     exit
370 KEY 5,"
380 ON KEY GOSUB 1000,2000,3000,400
__ 0,5000
390 FIELD#0,128 AS A$,128 AS B$
400 C$=DSKI$(DR,T,S)
410 X=10
420 X=VARPTR(A$):POKE &H9800,PEEK(X
__ +1):POKE &H9801,PEEK(X+2)
430 X=VARPTR(B$):POKE &H9802,PEEK(X
__ +1):POKE &H9803,PEEK(X+2)
440 DEFUSR=&H981A:X=USR(0)
450 LOCATE 11,18:PRINT "TRACK :"T:L
__ OCATE31,18:PRINT"SECTOR :"S
460 G=FN G(T,S):IF G<0 THEN 470 ELS
__ E LOCATE 59,18:H$=HEX$(G):PRINT
__ STRING$(2-LEN(H$),48)+H$" ("(G-
```

```
__ 0 USE (1 OR 2)";:DR$=INPUT$(1):D
__ R=VAL(DR$)
210 CLS:LOCATE 20,8:PRINT"IS THIS D
__ ISC A (B)ASIC OR (C)P/M DISC ";
220 DI$=INPUT$(1):IF DI$="B" OR DI$
__ ="b" THEN T=20 ELSE T=3
230 LOCATE 10,17:PRINTCHR$(180);STR
__ ING$(11,174);CHR$(168);SPC(7);CH
__ R$(180);STRING$(12,174);CHR$(168
__ ):LOCATE 10,18:PRINTCHR$(197);SP
__ C(11);CHR$(200);SPC(7);CHR$(197)
__ ;SPC(12);CHR$(200)
240 LOCATE 10,19:PRINTCHR$(197);SPC
__ (11);CHR$(200);SPC(7);CHR$(197);

__ SPC(12);CHR$(200)
250 LOCATE 10,20:PRINTSTRING$(13,17
__ 4);SPC(7);STRING$(14,174)
260 LOCATE 50,17:PRINTCHR$(180);STR
__ ING$(16,174);CHR$(168):LOCATE 50
__ ,18:PRINTCHR$(197);SPC(16);CHR$(
__ 200):LOCATE 50,19:PRINTCHR$(169)
__ ;STRING$(16,171);CHR$(170)
270 G=FN G(T,S):H$=HEX$(G):LOCATE 5
__ 1,18:PRINT"CP/M GR:";STRING$(2-L
__ EN(H$),48)+H$" ("(G-INT(G))*4+1"
__ )"
280 LOCATE 7,21:PRINT"S P E C T R A

__ V I D E O D I S C E D I T
__ O R ";CHR$(27)+"p";" drive
__ = ";DR$;CHR$(27)+"q";
290 LOCATE 11,18:PRINT "TRACK :"T:L
__ OCATE31,18:PRINT"SECTOR :"S
300 POKE &HD994,6:'DISC TURN OFF =
__ 30 SECS ; 1 FOR NORMAL
310 GOSUB 6000
__ 60 ELSE RETURN
2000 'edit
2010 YS=0:XS=6:L$="      00 11 22 3
__ 3  44 55 66 77  88 99 AA BB  CC
__ DD EE FF  0123456789ABCDEF
__ "
2020 LOCATE XS,YS,1:SCREEN,0
2030 Q$=INKEY$:IF Q$="" THEN 2030 E
__ LSE Q=ASC(Q$)
2040 IF Q=13 THEN GOSUB 1030:IF XS<
__ 58 THEN XS=6:GOTO 2020 ELSE XS=5
__ 8:GOTO 2020
2050 IF Q=27 OR Q=3 THEN LOCATE,,0:
__ SCREEN,1:RETURN
2060 IF Q=28 THEN GOSUB 1060:GOTO 2
__ 020
2070 IF Q=29 THEN GOSUB 1050:GOTO 2
__ INT(G))*4+1")"
470 KEYON:R$=INKEY$:IF R$="" THEN 4
__ 70 ELSE R=ASC(R$)
480 IF R=27 OR R=3 THEN 550:'Exit w
__ ith ESC or ^C
490 IF R=28 THEN S=S+1:IF S>17 THEN
__ S=1:'CURSOR -->
500 IF R=29 THEN S=S-1:IF S<1 THEN
__ S=17:'CURSOR <--
510 IF R=30 THEN T=T+1:IF T>39 THEN
__ T=0:'CURSOR ^
520 IF R=31 THEN T=T-1:IF T<0 THEN
__ T=39:'CURSOR v
530 IF R=93 THEN IFDI$="b"ORDI$="B"
__ THEN T=20:S=1ELSET=3:S=1:']=93
__ for DIRECTORY
540 GOTO 400
550 OUT52,0:POKE &HD994,2:SCREEN 0,
__ 1:LOCATE ,,1:CLS
560 KEY1," close:files "
570 KEY2," load "+CHR$(34)+"1:
580 KEY3," save "+CHR$(34)+"1:
590 KEY4," list "
600 KEY5," run"+CHR$(13)
610 ON ERROR GOTO 0
620 END
1000 '
1010 LOCATE 0,23:PRINT"Which drive
__ to log in ";:DR$=INPUT$(1):DR=VAL
__ (DR$):IF DR>2 THEN 1010
1020 RETURN 210
1030 YS=(YS+1)MOD 16:RETURN
1040 YS=(YS+15)MOD 16:RETURN
1050 XS=(XS+79)MOD 80:IF MID$(L$,XS
__ +1,1)=" " THEN 1050 ELSE RETURN
1060 XS=XS+1:IF XS=74 THEN XS=58:GO
__ TO 1030
1070 IF MID$(L$,XS+1,1)=" " THEN 10
3010 SCREEN,0:LOCATE0,23,1:PRINTCHR
__ $(27)"p Are you sure (Y/N)";
3020 IF ASC(INPUT$(1))MOD2=0 THEN 3
__ 050
3030 LOCATE 0,23:PRINT" Writing to
__ disk...";
3040 DSKO$ 1,T,S:FOR TD=0 TO 1000:N
__ EXT
3050 LOCATE0,0,0:SCREEN,1:PRINTCHR$
__ (27)"q";:RETURN
4000 'Exit
4010 RETURN 550
5000 '
5010 RETURN
6000 DATA A6,D2,26,D3,0,0,10,2E,0,1
__ 0,2E,2E,2E,2E,2E,2E,2E,2E,2E,2E,2E,2E,2E
6010 DATA 2E,2E,2E,2E,3E,0,32,8,98,
```

```
__ 020
2080 IF Q=30 THEN GOSUB 1040:GOTO 2
__ 020
2090 IF Q=31 THEN GOSUB 1030:GOTO 2
__ 020
2100 M=VAL("&H"+MID$(L$,XS+1,1)):M$
__ =MID$(L$,XS+2,1)
2110 IF XS<58 THEN 2150
2120 IF YS<8 THEN MID$(A$,YS*16+M+1
__ ,1)=Q$ ELSE MID$(B$,(YS-8)*16+M+
__ 1,1)=Q$
2130 PRINT Q$;:LOCATE M*3+M/4+6:PRI
__ NTRIGHT$("0"+HEX$(ASC(Q$)),2);
2140 GOSUB 1060:GOTO 2020
2150 IF INSTR("0123456789ABCDEFabcd
__ ef",Q$)=0 THEN 2020 ELSE Q=VAL("
__ &H"+Q$)
2160 IF YS<8 THEN A=ASC(MID$(A$,YS*
__ 16+M+1,1)) ELSE A=ASC(MID$(B$,(Y
__ S-8)*16+M+1,1))
2170 IF M$<>" "THEN A=(A AND 15)+Q*
__ 16 ELSE A=(A AND 240)+Q
2180 IF YS<8 THEN MID$(A$,YS*16+M+1
__ ,1)=CHR$(A) ELSE MID$(B$,(YS-8)*
__ 16+M+1,1)=CHR$(A)
2190 PRINT HEX$(Q);
2200 IF A<32 OR A>126 THEN A=46
2210 LOCATE M+58:PRINT CHR$(A);:GOS
__ UB 1060:GOTO 2020
3000 'Write
```

```
__ 32,9,98,3E,0,32,4,98,3E,0,CD,D9,98,3A
6020 DATA 4,98,CD,D9,98,3E,20,CD,12
__ ,99,3E,20,CD,12,99,21,A,98,11,B,98,1,F
6030 DATA 0,36,0,ED,B0,3E,0,32,6,98
__ ,3A,4,98,47,3A,6,98,80,2A,0,98,85,6F,7C
6040 DATA CE,0,67,7E,32,7,98,CD,D9,
__ 98,3E,20,CD,12,99,3A,6,98,E6,3,FE,3,20
6050 DATA 5,3E,20,CD,12,99,3A,7,98,
__ FE,20,38,6,FE,7F,30,2,18,2,3E,2E,32,7
6060 DATA 98,21,A,98,3A,6,98,85,6F,
__ 7C,CE,0,67,3A,7,98,77,3A,6,98,3C,32,6
6070 DATA 98,CB,67,28,A8,6,10,21,A,
__ 98,7E,CD,12,99,23,10,F9,3E,0,32,8,98,3A
6080 DATA 9,98,3C,32,9,98,3A,4,98,C
__ 6,10,32,4,98,CB,7F,28,A,2A,2,98,11,80
6090 DATA FF,19,22,0,98,A7,C2,27,98
__ ,C9,F5,E6,F0,CB,3F,CB,3F,CB,3F,CB,3F,21
6100 DATA 2,99,85,6F,7C,CE,0,67,7E,
__ CD,12,99,F1,E6,F,21,2,99,85,6F,7C,CE,0
6110 DATA 67,7E,CD,12,99,C9,30,31,3
__ 2,33,34,35,36,37,38,39,41,42,43,44,45
6120 DATA 46,C5,D5,E5,F5,3A,9,98,6F
__ ,26,0,CB,25,CB,14,CB,25,CB,14,CB,25,CB
6130 DATA 14,CB,25,CB,14,E5,D1,CB,2
__ 5,CB,14,CB,25,CB,14,19,E5,D1,21,0,F0,19
6140 DATA 3A,8,98,85,6F,7C,CE,0,67,
__ F3,3E,FF,D3,58,F1,D6,20,77,3E,0,D3,58
6150 DATA FB,3A,8,98,3C,32,8,98,E1,
__ D1,C1,C9
6160 RESTORE 6000:FOR I=&H9800TO &H995E:
__ READA$:POKE I,VAL("&H"+A$):NEXT:RETURN
```

```
100 REM Fractal tree Steve Lacey
110 SCREEN 1
120 SL=60:          'stork length
130 RF=.65:       'reduction factor
140 AN=1.05:        'angle
150 DIM A(256),X(256),Y(256)
160 A(0)=0:X(0)=128:Y(0)=191:R=0
170 PSET (X(0),Y(0))
180 X(0)=X(0)+SIN(A(0))*SL
190 Y(0)=Y(0)-COS(A(0))*SL
200 LINE -(X(0),Y(0))
210 SL=SL*RF
220 FOR G=1 TO 8:FOR H=0 TO R:A=A(H
__ ):X=X(H):Y=Y(H):R=R+1:A(R)=A+AN:
__ X(R)=X+SIN(A(R))*SL:Y(R)=Y-COS(A
__ (R))*SL:LINE (X,Y)-(X(R),Y(R)):A
__ (H)=A-AN:X(H)=X+SIN(A(H))*SL:Y(H
__ )=Y-COS(A(H))*SL:LINE (X,Y)-(X(H
__ ),Y(H)):NEXT H:SL=SL*RF:NEXT G
230 A$=INPUT$(1)
```

```
100 REM Dandelion, derived from
110 REM FractalTree by Steve Lacey
120 DEFSNG A-Z:DEFINT G,H:SCREEN 1
130 SL=70: 'stalk length
140 RF=.32:'reduction factor
150 B=6   :'branches (max 10)
160 G1=4  :'generations
170 AN=5.5/B
220  FOR H=0 TO R
230   A=A(H):X=X(H):Y=Y(H)
240    FOR I=0 TO B-1
250    Z=H:IF I THEN R=R+1:Z=R
260    A(Z)=A+(AN*I)-(AN*(B-1)*.5)
270    X(Z)=X+SIN(A(Z))*SL*.8
280    Y(Z)=Y-COS(A(Z))*SL
290    LINE (X,Y)-(X(Z),Y(Z))
```

```
180 DIM A(B^G1),X(B^G1),Y(B^G1)
190 A(0)=0:X(0)=128:Y(0)=96:R=0
200 LINE (128,191)-(X(0),Y(0))
210 FOR G=1 TO G1
300   NEXT
310  NEXT
320  SL=SL*RF
330 NEXT
340 A$=INPUT$(1)
```

This next one finds and highlights found strings in SV basic and MBASIC. Only to be used on ASCII files !!

```
100 REM basic FIND3 for conversion
__ to MBASIC
110 CLEAR 3000
120 ESC$=CHR$(27):IV$=ESC$+"p":NV$=
__ ESC$+"q":RE$="C"
130 N=0:'DIM A$(20)
140 PRINT CHR$(12)
150 INPUT "File to search ";F$
160 INPUT "String to search for";S$
170 PRINT CHR$(12)
180 OPEN F$ FOR INPUT AS #1
190 'OPEN "I",1,F$:'MBASIC
200 IF EOF(1) THEN CLOSE:PRINT"eof"
__ :END
210 LINE INPUT #1,A$:P=INSTR(A$,S$)
220 IF P > 0  THEN GOSUB 270
230 IF N > 9 THEN PRINT STRING$(15,
__ 32)+"MORE ?...ESC TO QUIT":RES=I
__ NPUT$(1):N=0:CLS:FL=0
240 IF ASC(RE$)=27 THEN CLOSE:END
250 IF FL THEN PRINT A$:N=N+1:GOTO
__ 200
260 GOTO 200
270 PRINT LEFT$(A$,P-1);IV$;MID$(A$
__ ,P,LEN(S$));NV$;RIGHT$(A$,LEN(A$
__ )+1-(P+LEN(S$)))
280 FL=-1
290 RETURN 200
```

Thats all folks !!

IPL.ED

```
10 CLEAR2000:IV$=CHR$(27)+"p":GOSUB870
20 NM$=CHR$(27)+"q":GOSUB860
30 REM FOR VARIBLES
40 LOCATE0,3
50 PRINT"$                MAIN MENU"
60 PRINT"$                ,,,,,,,,,"
70 PRINT
80 PRINT"$       1) LOG IN NEW DISK"
90 PRINT"$       2) NEW IPL LINE"
100 PRINT"$      3) EDIT PRESENT IPL LINE":
PRINT"$      4) SAVE IPL LINE TO DISK"
110 PRINT"$      5) EXIT THIS PROGRAM"
120 PRINT:PRINT"$     ENTER NUMBER OF YOUR C
HOICE:"
130 LOCATE,,0:POKE&HF543,38
140 FORI=1TOLEN(IP$)
150 B$=MID$(IP$,I,1):B=ASC(B$)
160 IF(B>126ANDB<160)ORB<32ORB>215THENPRINTI
V$;RIGHT$(STR$(B),LEN(STR$(B))-1);NM$;:ELSEP
RINTB$;
170 NEXTI:POKE&HF543,40
180 LOCATE34,12,1:A$=INPUT$(1):IFA$<"1"ORA$>
"5"THEN180
190 PRINTA$
200 A=VAL(A$):ONAGOTO220,490,270,240,210
210 CLS:PRINT"FINISHED."
220 END
230 GOSUB870:LOCATE2,11:PRINT"PUT DISK TO LO
G IN IN DRIVE A"
240 LOCATE2,13:PRINT" PRESS ANY KEY:";:A$=IN
PUT$(1)
250 GOSUB860:GOSUB870
260 GOTO40
270 C=1:GOSUB870:POKE&HF543,38
280 LOCATE1,5:GOSUB380
290 LOCATE1,5:LINEINPUT"";IP$
300 POKE&HFA06,0:IP$=LEFT$(IP$,255)
310 FORI=1TOLEN(IP$):B$=MID$(IP$,I,1)
320 IFB$="@"THEN350
330 NEXT
340 IPLIP$:GOSUB870:GOTO40
350 LOCATE1,15:PRINT"WHICH CHAPACTER CODE DO
ES '@' %o';C;"     REPPESENT? :";
360 LINEINPUT"";A$:A=VAL(A$):IFA<OORA>255THE
N350
370 C=C+1:MID$(IP$,I,1)=CHR$(A):GOTO330
380 LOCATE,,0:POKE&HF543,38
390 X2=1:Y2=15:FORI=1TOLEN(IP$)
400 B$=MID$(IP$,I,1):B=ASC(B$)
410 IF(B>126ANDB<160)ORB<32ORB>215)ANDB>9TH
EN440ELSEPRINTB$;
420 NEXTI:LOCATE,,1:POKE&HFA06,249
430 RETURN
440 X1=POS(0):Y1=CSRLIN:LOCATEX2,Y2
450 PRINTIV$;RIGHT$(STR$(B),LEN(STR$(B))-1);
NM$;
460 X2=X2+4:IFX2>37THENX2=1:Y2=Y2+1
470 LOCATEX1,Y1:PRINTIV$;"@";NM$;
480 GOTO420
490 GOSUB870:IP$="":IP=255:POKE&HF543,38
500 LOCATE1,3:PRINT"WHAT WOULD YOU LIKE THE
IPL LINE TO   DO?"
510 LOCATE1,6:PRINT"CLEAR THE SCREEN? (Y/N)
";:A$=INPUT$(1):IFA$="Y"ORA$="y"THENIP$="CLS
":":IP=251
520 LOCATE1,6:PRINTCHR$(27)+"1";"TURN ON CAP
S? (Y/N)";:A$=INPUT$(1):IFA$="Y"ORA$="y"THEN
IP$=IP$+"DEFUSR=16359:Y=USR(0):":IP=IP-22
530 LOCATE1,6:PRINTCHR$(27)+"1";"CHANGE SCRE
EN WIDTH? (Y/N)";:A$=INPUT$(1):IFA$="Y"ORA$=
"y"THENIP$=IP$+"WIDTH":IP=IP-5
540 IFA$="Y"ORA$="y"THENLOCATE1,6:PRINTCHR$(
27)+"1";"WIDTH (3)9 , (4)0 , OR (8)0:";:A$=I
NPUT$(1):IFA$()"3"ANDA$()"4"ANDA$()"8"THENAS
="Y":GOTO540ELSEIP$=IP$+A$+"0:":IP=IP-3
550 LOCATE1,6:PRINTCHR$(27)+"1";"CHANGE SCRE
EN COLOUR? (Y/N)";:A$=INPUT$(1):IFA$="Y"ORA$
="y"THENIP$=IP$+"COLOR":IP=IP-5
560 IFA$="Y"ORA$="y"THENLOCATE1,5:PRINTCHR$(
27)+"1";"ENTER COLOURS (INK,PAPER):";:LINEIN
PUT$:A$=LEFT$(A$,4):IP$=IP$+A$+":":IP=IP-(L
EN(A$)+1)
570 LOCATE1,6:PRINTCHR$(27)+"1";"CHANGE KEY
1? (Y/N)";:A$=INPUT$(1):IFA$="Y"ORA$="y"THEN
IP$=IP$+"KEY1,"+CHR$(34):IP=IP-6
580 IFA$="Y"ORA$="y"THENLOCATE1,6:PRINTCHR$(
27)+"1";"TO WHAT? ENTER NEW FUNCTION:";:LINE
INPUTA$:A$=LEFT$(A$,15):IP$=IP$+A$+CHR$(34)+
":":IP=IP-(LEN(A$)+2)
590 LOCATE1,6:PRINTCHR$(27)+"1";"CHANGE KEY
2? (Y/N)";:A$=INPUT$(1):IFA$="Y"ORA$="y"THEN
IP$=IP$+"KEY2,"+CHR$(34):IP=IP-6
600 IFA$="Y"ORA$="y"THENLOCATE1,6:PRINTCHR$(
27)+"1";"TO WHAT? ENTER NEW FUNCTION:";:LINE
INPUTA$:A$=LEFT$(A$,15):IP$=IP$+A$+CHR$(34)+
":":IP=IP-(LEN(A$)+2)
610 LOCATE1,6:PRINTCHR$(27)+"1";"CHANGE KEY
3? (Y/N)";:A$=INPUT$(1):IFA$="Y"ORA$="y"THEN
IP$=IP$+"KEY3,"+CHR$(34):IP=IP-6
620 IFA$="Y"ORA$="y"THENLOCATE1,6:PRINTCHR$(
27)+"1";"TO WHAT? ENTER NEW FUNCTION:";:LINE
INPUTA$:A$=LEFT$(A$,15):IP$=IP$+A$+CHR$(34)+
":":IP=IP-(LEN(A$)+2)
630 LOCATE1,6:PRINTCHR$(27)+"1";"CHANGE KEY
4? (Y/N)";:A$=INPUT$(1):IFA$="Y"ORA$="y"THEN
IP$=IP$+"KEY4,"+CHR$(34):IP=IP-6
640 IFA$="Y"ORA$="y"THENLOCATE1,6:PRINTCHR$(
27)+"1";"TO WHAT? ENTER NEW FUNCTION:";:LINE
INPUTA$:A$=LEFT$(A$,15):IP$=IP$+A$+CHR$(34)+
":":IP=IP-(LEN(A$)+2)
650 LOCATE1,6:PRINTCHR$(27)+"1";"CHANGE KEY
5? (Y/N)";:A$=INPUT$(1):IFA$="Y"ORA$="y"THEN
IP$=IP$+"KEY5,"+CHR$(34):IP=IP-6
660 IFA$="Y"ORA$="y"THENLOCATE1,6:PRINTCHR$(
27)+"1";"TO WHAT? ENTER NEW FUNCTION:";:LINE
INPUTA$:A$=LEFT$(A$,15):IP$=IP$+A$+CHR$(34)+
":":IP=IP-(LEN(A$)+2)
670 LOCATE1,6:PRINTCHR$(27)+"1";"CHANGE KEY
6? (Y/N)";:A$=INPUT$(1):IFA$="Y"ORA$="y"THEN
IP$=IP$+"KEY6,"+CHR$(34):IP=IP-6
680 IFA$="Y"ORA$="y"THENLOCATE1,6:PRINTCHR$(
27)+"1";"TO WHAT? ENTER NEW FUNCTION:";:LINE
INPUTA$:A$=LEFT$(A$,15):IP$=IP$+A$+CHR$(34)+
":":IP=IP-(LEN(A$)+2)
690 LOCATE1,6:PRINTCHR$(27)+"1";"CHANGE KEY
7? (Y/N)";:A$=INPUT$(1):IFA$="Y"ORA$="y"THEN
IP$=IP$+"KEY7,"+CHR$(34):IP=IP-6
700 IFA$="Y"ORA$="y"THENLOCATE1,6:PRINTCHR$(
27)+"1";"TO WHAT? ENTER NEW FUNCTION:";:LINE
INPUTA$:A$=LEFT$(A$,15):IP$=IP$+A$+CHR$(34)+
":":IP=IP-(LEN(A$)+2)
710 LOCATE1,6:PRINTCHR$(27)+"1";"CHANGE KEY
8? (Y/N)";:A$=INPUT$(1):IFA$="Y"ORA$="y"THEN
IP$=IP$+"KEY8,"+CHR$(34):IP=IP-6
720 IFA$="Y"ORA$="y"THENLOCATE1,6:PRINTCHR$(
27)+"1";"TO WHAT? ENTER NEW FUNCTION:";:LINE
INPUTA$:A$=LEFT$(A$,15):IP$=IP$+A$+CHR$(34)+
":":IP=IP-(LEN(A$)+2)
730 LOCATE1,6:PRINTCHR$(27)+"1";"CHANGE KEY
9? (Y/N)";:A$=INPUT$(1):IFA$="Y"ORA$="y"THEN
IP$=IP$+"KEY9,"+CHR$(34):IP=IP-6
740 IFA$="Y"ORA$="y"THENLOCATE1,6:PRINTCHR$(
27)+"1";"TO WHAT? ENTER NEW FUNCTION:";:LINE
INPUTA$:A$=LEFT$(A$,15):IP$=IP$+A$+CHR$(34)+
":":IP=IP-(LEN(A$)+2)
750 IFIP<247HEN770ELSELOCATE1,6:PRINTCHR$(27
)+"1";"CHANGE KEY 10? (Y/N)";:A$=INPUT$(1):I
FA$="Y"ORA$="y"THENIP$=IP$+"KEY10,"+CHR$(34)
:IP=IP-7
760 IFA$="Y"ORA$="y"THENLOCATE1,6:PRINTCHR$(
27)+"1";"TO WHAT? ENTER NEW FUNCTION:";:LINE
INPUTA$:A$=LEFT$(A$,15):IP$=IP$+A$+CHR$(34)+
":":IP=IP-(LEN(A$)+2)
770 IFIP<97HEN810ELSELOCATE1,6:PRINTCHR$(27)
+"1";"PRINT A MESSAGE? (Y/N)";:A$=INPUT$(1):
IFA$="Y"ORA$="y"THENIP$=IP$+"PRINT"+CHR$(34)
:IP=IP-6
780 IFA$="Y"ORA$="y"THENLOCATE1,6:PRINTCHR$(
27)+"1";"ENTER MESSAGE (NO SPEACH MARKS PLEA
SE):";:LINEINPUTA$:A$=LEFT$(A$,IP-1):IP$=IP$
+A$+CHR$(34):IP=IP-(LEN(A$)+1)
790 IFIP<87HEN810ELSELOCATE1,6:PRINTCHR$(27)
+"1";"LOAD A PROGRAM? (Y/N)";:A$=INPUT$(1):I
FA$="Y"ORA$="y"THENIP$=IP$+"LOAD"+CHR$(34)+"
":":IP=IP-7
800 IFA$="Y"ORA$="y"THENLOCATE1,6:PRINTCHR$(
27)+"1";"ENTER PROGRAM NAME (NO DRIVE NO. PL
EASE):";:LINEINPUTA$:A$=LEFT$(A$,IP-1):IP$=I
P$+A$+CHR$(34):IP=IP-(LEN(A$)+1)
810 IFIP<77HEN830ELSELOCATE1,6:PRINTCHR$(27)
+"1";"RUN A PROGRAM? (Y/N)";:A$=INPUT$(1):IF
A$="Y"ORA$="y"THENIP$=IP$+"PUN"+CHR$(34)+":
":IP=IP-6
820 IFA$="Y"ORA$="y"THENLOCATE1,6:PRINTCHR$(
27)+"1";"ENTER PROGRAM NAME (NO DRIVE NO. PL
EASE):";:LINEINPUTA$:A$=LEFT$(A$,IP-1):IP$=I
P$+A$+CHR$(34):IP=IP-(LEN(A$)+1)
830 IP$=LEFT$(IP$,255):PRINT:PRINT"NOW SAVE
IPL LINE"
840 FORI=1TO1000:NEXT:POKE&HF542,40:POKE&HFA
```

```
06,255:GOSUB870:GOTO40
850 END
860 IP$=DSKI$(1,20,14):RETURN
870 CLS:WIDTH40:COLOR15,4:LOCATE,,0
880 PRINT"W0000000000000000000000000000000000
00000B";
890 PRINT"S          IPL EDITOR     MAW '88
    S";
900 PRINT" 000000000000000000000000000000
00000#"
910 FORX=3TO19
920 VPOKE40*X,243
930 VPOKE40*X+39,243
940 NEXT
950 LOCATE0,20:PRINT"C0000000000000000000000000000
0000000000000000D";
960 LOCATE,,1:RETURN
```